SE301 ADVANCED PROGRAMMING AND DESIGN · PROJECT 2026-27

Singapore Vertical Farming Research Management System

Legacy Refactoring & Modernisation Report (README)

Group	[Group number]
Member 1	[Full name]
Member 2	[Full name]
Member 3	[Full name]

1. Legacy System Analysis

SVFRMS runs an indoor vertical farm. Each growing chamber holds one crop. The system reads the chamber's sensors (temperature, humidity, soil moisture, light and nutrients), compares the readings with what the crop needs, raises alerts when something is wrong, switches on devices such as fans and pumps to fix it, and writes a report at the end of the day.

The program we were given is one Java class, `SVFRMS` (393 lines). Its main parts are:

Part	What it does in the legacy code
Crop rules	`processSensor()` holds every crop's limits as hard-coded `if` statements.
Chamber simulation	Four `ChamberThread`s (Tomato, Strawberry, Lettuce, Chili) each produce 20 rounds of random readings.
Device control	`addControlRequest()` puts `"CH1|Cooling Fan"` strings on a shared list; one `ControlThread` takes them off and calls `activateControl()`.
Alerts and logging	`handleEvent(type, ...)` uses a string `type` (`"LOG"`, `"CREATE_ALERT"`, `"TRACK_ACTION"` ...) to decide what to do.
Reports	`generateDailyReport()` and `archiveAlerts()` run on two threads; `saveArchiveReport()` writes `archive/daily_report.txt`.
Shared data	Everything lives in `public static` fields: lists, maps, counters and lock objects.

2. Code Analysis and Limitations

2.1 Design problems

Problem	Example from the legacy code	Principle broken
One class does everything	Sensor logic, rules, devices, threads, alerts, reports and file writing are all in `SVFRMS`. A change to the report format and a change to a crop limit both edit the same file.	SRP
Crop limits are code	`if(crop.equals("Tomato")){ if(value>28){...} if(value<22){...} } else if(crop.equals("Strawberry")) ...` Adding Kale means editing `processSensor()` in several places.	OCP
Copy-pasted blocks	The same line `handleEvent("TRACK_ALERT"...); handleEvent("CREATE_ALERT"...); addControlRequest(...)` appears about 25 times with only the text changed.	DRY
String "flag" arguments	`handleEvent("CREATE_ALERT", ...)` and `activateControl(chamber, "Cooling Fan")` switch on strings. A typo compiles but silently does nothing.	OCP, flag-argument smell
Global mutable state	`public static ArrayList<String> alerts`, `public static int totalWaterUsage` ... Any class can change anything, and tests cannot run in isolation.	Information hiding, DIP
Depends on concrete classes	`new Random()`, `System.out` and `Files` are used directly inside the logic, so nothing can be replaced (for example with a test stub).	DIP
Inheritance used for threads	`ChamberThread extends Thread`: the task is tied to how it runs and inherits many methods it never uses. A `Runnable` / `Callable` run by an executor is simpler.	Composition over inheritance
Rules do not match the spec	Only Tomato has a soil rule; only Tomato and Strawberry have humidity rules; light and nutrient limits ignore the crop; Spinach and Cucumber rules exist but never run. No alert severity; Heating Unit, Ventilation and Shade Controller are missing; only 1 of the 5 reports exists.	Hidden by duplication
Dead code and swallowed errors	`waterPumpLock`, `nutrientLock`, `irrigationCycle()`, `feedingCycle()`, `showAlerts()` are never used. `catch(Exception e){}` hides interrupts.	Clean code

Spec questions (4.2): (i) yes, one class has many unrelated jobs; (ii) yes, every new crop needs code changes; (iii) there is no Square/Rectangle-style LSP break, but extending `Thread` is inheritance that is not needed; (iv) the class's huge static API, and `handleEvent` doing six jobs in one method, should be split; (v) everything depends on concrete classes.

2.2 Concurrency problems

#	Problem	Why it goes wrong
C1	Shared counters, e.g. `totalWaterUsage += 2`	`+=` is read, add, write. Two threads can read the same old value and one update is lost.
C2	`ArrayList` written by many threads (`alerts` , `sensorLogs`)	`ArrayList` is not thread-safe. When we ran the legacy code, a chamber thread crashed with `ArrayIndexOutOfBoundsException` inside `sensorLogs.add()` and that chamber stopped being monitored.
C3	Check-then-act on `HashMap` in `getChamberState()`	`if(!containsKey) put(...)` : two threads can both see "missing" and create two different objects.
C4	`ChamberState` fields	The chamber thread writes new readings while the control thread does `temperature - 5` on the same fields, with no lock.
C5	`ChamberSummary` counters	`summary.alerts++` runs on several threads with no lock.
C6	Nested locks `alertLock` then `reportLock`	Holding two locks at once is the setup for a deadlock: if any code takes them in the other order, both threads wait forever.
C7	Report and archive threads race	The two threads start together, so "Active Alerts" in the report changes from run to run.
C8	Hand-made `wait()/notifyAll()` queue	Interrupts are swallowed, and if the control thread dies the chamber threads block forever on a full queue.

3. Proposed Design

The goal is to keep what the system does but make it easy to change. The key changes are:

- **Split by responsibility.** The monolith became 12 small packages (52 classes). Each class has one job, so a change touches one place.
- **Crops are data, not code.** Crop limits live in `crops.csv` and chamber assignments in `chambers.csv` . A `CropProfile` must have a range for every sensor.
- **One rule instead of 25 copies.** A single `ThresholdRule` reads the limit from the crop profile, so it never names a crop. A `SensorFailureRule` catches impossible readings.
- **Devices are objects.** Each `Device` knows its effect (e.g. Cooling Fan: temperature −5, not below 18 C) and its resource cost. No more `if(control.equals(...))` .
- **Events instead of a string switch.** Alerts and device activations are published on an `EventBus` . Storing, counting and printing are separate listeners.
- **No global state.** Each `FarmSystem` owns its own stores, so tests can create one, run it and check it.
- **Thread-safe by design.** Shared data is either immutable or guarded inside its own class. Threads are run by an `ExecutorService` .

Behaviour aligned with the specification. We kept every legacy feature but fixed the places where the legacy code did not follow the spec: every crop now uses all five ranges; alerts have INFO / WARNING / CRITICAL severity (based on how far the value is outside the range); low temperature starts the Heating Unit, high light starts the Shade Controller, high humidity starts the Dehumidifier and Ventilation System; impossible readings raise a Sensor Failure alert; and all five reports are produced. Water, energy and nutrient use is now counted when a device actually runs.

3.1 How SOLID is applied

Principle	Where it shows in the new code
SRP	`AlertStore` only stores alerts, `ConsoleEventPrinter` only prints, `ArchiveReportWriter` only writes the file, `ResourceMeter` only counts usage.
OCP	New crop = new CSV row. New device, rule or report = new class registered in one list (`StandardDevices` , `StandardRules` , `ReportGenerator`). Existing classes are not edited.

LSP	Every `SensorRule`, `Device` and `Report` follows its interface fully, so any of them can be swapped in. Threads now use `Callable` / `Runnable` instead of extending `Thread`.
ISP	Small interfaces: `SensorSource` and `SensorRule` have one method; `CropRepository` is read-only. Event listeners are plain `Consumer`s, so each listener only handles the event it needs.
DIP	`ChamberMonitor` depends on `SensorSource`, `RuleEngine` and `Device` abstractions. Only `Main` picks the concrete classes and passes them in.

4. Final Architecture

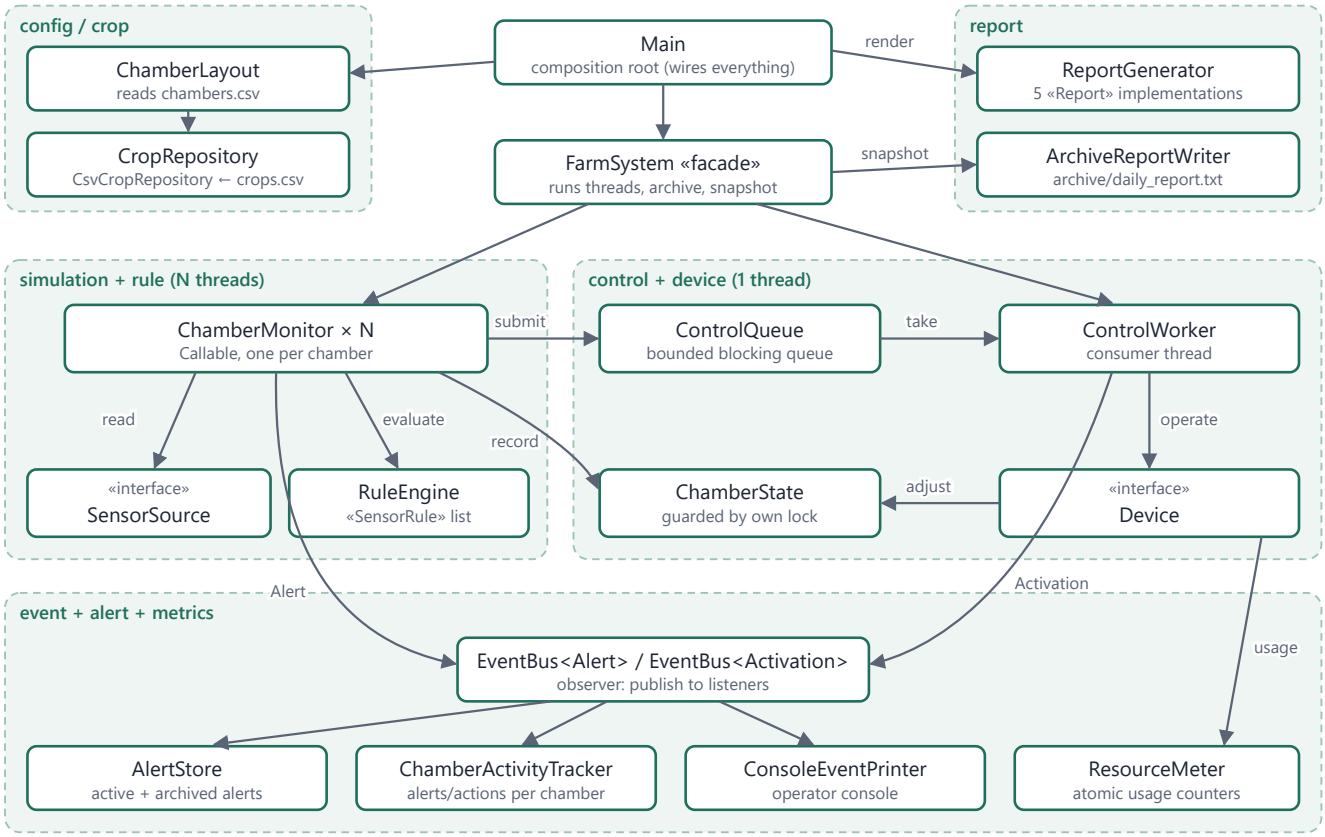


Figure 1 – Component diagram. Arrows show "uses / sends to".

Package	Contents
`model`	`Range`, `SensorType`, `SensorReading`, `CropProfile` (immutable), `Chamber`, `ChamberState` (thread-safe)
`config`, `crop`	`CsvFile` (one shared CSV reader), `ChamberLayout`, `CropRepository` + `CsvCropRepository`
`rule`	`SensorRule`, `ThresholdRule`, `SensorFailureRule`, `SeverityPolicy`, `RuleEngine`, `StandardRules`
`device`, `control`	`Device`, `EnvironmentalDevice`, `Effect`, `StandardDevices`; `ControlRequest`, `ControlQueue`, `ControlWorker`, `Activation`
`event`, `alert`, `metrics`	`EventBus`; `Alert`, `AlertType`, `Severity`, `AlertStore`; `ResourceMeter`, `ChamberActivityTracker`, `ReadingStatistics`
`simulation`, `report`, `app`	`SensorSource`, `RandomSensorSource`, `ChamberMonitor`; five reports, `ReportGenerator`, `ArchiveReportWriter`; `Main`, `FarmSystem`, `ConsoleEventPrinter`

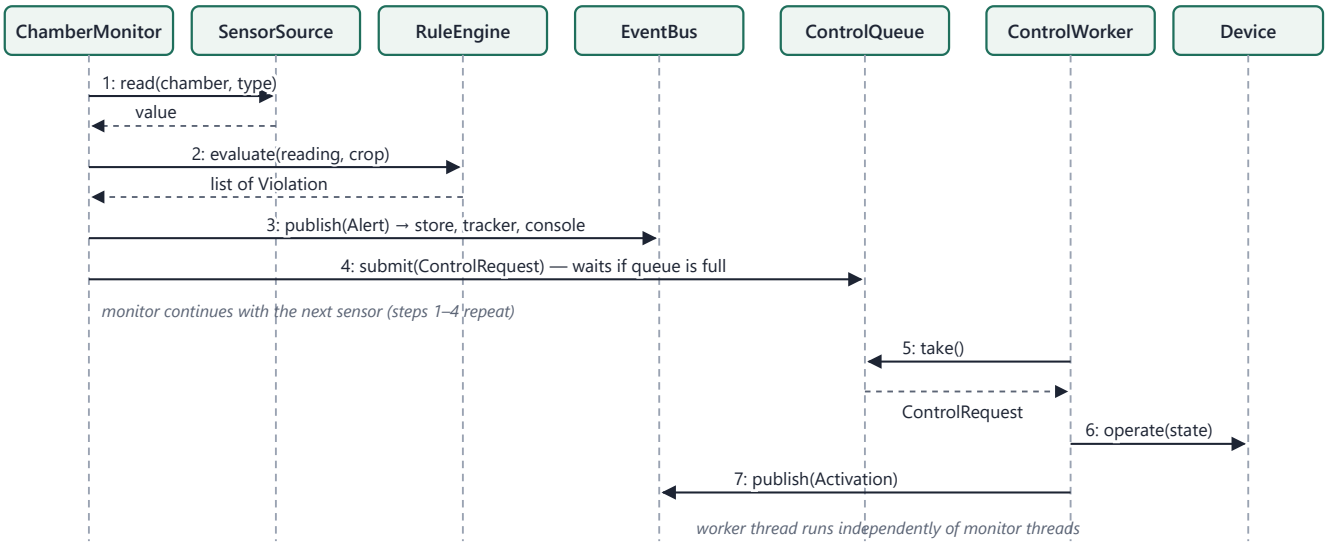


Figure 2 – Sequence diagram: from one sensor reading to a device running.

How a run works. `Main` loads the crops and chambers and builds a `FarmSystem`. `FarmSystem` starts one `ControlWorker` and one `ChamberMonitor` per chamber on a thread pool. Monitors read sensors, check rules, publish alerts and queue device requests (Figure 2). When every monitor has finished, `FarmSystem` closes the queue, waits for the worker to finish the remaining requests, then archives the alerts. Finally a read-only `FarmSnapshot` is taken and passed to the five reports and the archive writer. These steps run one after another, so the report is always consistent.

5. Concurrency Fixes

#	Fix	Where
C1	Counters are `AtomicLong`s kept in a `ConcurrentHashMap`. `addAndGet` is one atomic step, so no update is lost.	`ResourceMeter`, `ReadingStatistics`
C2	No shared `ArrayList` is written without a lock. Alerts go into `AlertStore`, whose methods are `synchronized`. The unused per-reading log list was replaced by two atomic counters.	`AlertStore`, `ReadingStatistics`
C3	Each `Chamber` owns its `ChamberState`, so the lookup map is gone. Where a map is still needed, `computeIfAbsent` / `compute` create and update entries atomically.	`Chamber`, `ResourceMeter`
C4	`ChamberState` keeps its values private and every method is `synchronized`. `adjust()` does read-change-write as one step.	`ChamberState`
C5	The summary is an immutable `ChamberActivity` record, replaced with `ConcurrentHashMap.compute()`, which is atomic per chamber.	`ChamberActivityTracker`
C6	No thread ever holds two locks. Each class guards only its own data with one lock, and archiving is a single `synchronized` method. This removes the "hold and wait" condition needed for a deadlock.	`AlertStore`
C7	Archive and report now run in a fixed order after all threads finish (`invokeAll` + `Future.get()`), and reports read an immutable snapshot.	`FarmSystem`, `FarmSnapshot`
C8	`ArrayBlockingQueue` handles waiting and waking. Shutdown is an "end" marker placed after pending requests, so none are lost. Interrupts are restored, not swallowed. The worker catches errors per request so it never dies and leaves producers stuck.	`ControlQueue`, `ControlWorker`

Threads are created by an `ExecutorService`; monitors are `Callable`s, so any error inside a thread is returned through its `Future` instead of being lost. `RandomSensorSource` uses `ThreadLocalRandom` so threads do not compete for one shared `Random`.

> **Proof by tests.** `ConcurrencyTest` runs 8 threads × 5,000 operations against `ResourceMeter`, `AlertStore` (adding while archiving), `ChamberState` and `ChamberActivityTracker`, and checks that the totals are exact.
> `FarmSystemTest` runs 4 chambers with a queue of size 1 (so producers must block) under a 10-second timeout to show there is no deadlock, and checks that every alert caused exactly one device activation.

6. Extensibility and Maintainability

Change	What you do	Existing code edited?
Add a crop (e.g. Kale)	Add one line to `crops.csv`: name and min/max for each sensor. Kale, Bell Pepper, Bok Choy and Eggplant were added this way, and CH5 now grows Kale.	None
Remove a crop	Delete its line from `crops.csv` (and any chamber using it in `chambers.csv`).	None
Change a crop's limits or a chamber's crop	Edit the CSV. A `crops.csv` next to `run.jar` overrides the built-in one, so no rebuild is needed.	None
Add a device	Create an `EnvironmentalDevice` with its effects and cost (or a new `Device` class) and use it in a rule.	`StandardDevices` list only
Add a rule or alert type	Implement `SensorRule` and add it to `StandardRules`.	`StandardRules` list only
Add a report	Implement `Report` and add it to `ReportGenerator.standard()`.	One list only
Use real sensors	Implement `SensorSource` and pass it in `Main`.	`Main` only
Send alerts somewhere new (SMS, file)	Subscribe a new listener with `farm.onAlert(...)`.	`Main` only

Maintainability also comes from: small classes with clear names; immutable records for data passed between threads; validation in constructors (a crop missing a range, or an unknown crop in `chambers.csv`, fails at start-up with a clear message); and 30 JUnit tests covering boundary values, CSV loading and thread safety.

7. Design Patterns

Pattern	Where	Problem it solves / why we chose it
Strategy	`SensorRule`, `Device`, `SeverityPolicy`, `Report`	Replaces long `if/else` chains on crop and device names. Each behaviour is its own object behind one interface, so new behaviour is added without editing old code (OCP).
Observer	`EventBus` + listeners	Replaces `handleEvent("CREATE_ALERT"...)`. The monitor just publishes an alert; storing, counting and printing are separate listeners that can be added or removed freely.
Command	`ControlRequest`	A device request becomes a typed object (chamber + device) that can be queued and run later by another thread, instead of a `"CH1|Cooling Fan"` string that had to be split.
Producer–Consumer	`ControlQueue`, `ControlWorker`	Monitors (producers) and the device worker (consumer) run at their own speed. A bounded queue limits pending work, and the Java library handles the waiting safely.
Facade	`FarmSystem`	Hides the wiring of buses, stores, queue and thread pool behind three calls: `runMonitoring()`, `archiveAlerts()`, `snapshot()`.
Repository	`CropRepository`	The rest of the system asks for crops without knowing they come from a CSV file. Storage could change (e.g. to a database) without touching the rules.
Dependency Injection	`Main`	One place creates concrete classes and passes them in through constructors. This is what lets tests use a fixed `SensorSource` instead of random values.

Patterns we chose not to use. *Singleton*: the legacy code's global static state is exactly the "singleton abuse" problem, so we removed it instead of adding more. *State*: devices are simply activated, with no on/off lifecycle, so a State pattern would add classes without solving a real problem (KISS).

How to run

`java -jar run.jar` (Java 25). The console shows alerts with severity and device actions, then the five reports; the full report is saved to `archive/daily_report.txt`. To build and test from source: `mvn package`.